Socket Programming — Lab Exam Algorithms

Socket Programming — Server Algorithm

1. Start the server program.

2. Create a TCP socket using IPv4.

```
s = socket(AF_INET, SOCK_STREAM, 0);
```

3. Configure the server address with IPv4, port `8080`, and allow connections from any local interface.

```
a.sin_family = AF_INET;
a.sin_port = htons(8080);
a.sin_addr.s_addr = INADDR_ANY;
```

4. Bind the socket to the specified address and port.

```
bind(s, (struct sockaddr*)&a, sizeof(a));
```

5. Put the socket into listening mode and wait for a client connection.

```
listen(s, 5);
```

6. Accept a connection from the client.

```
c = accept(s, NULL, NULL);
```

7. Receive the message from the client.

```
read(c, buf, sizeof(buf));
```

8. Display the message received from the client.

```
printf("Client: %s\n", buf);
```

9. Send `"Hello Client"` as a reply to the client.

```
send(c, "Hello Client", 12, 0);
```

10. Close the client socket and server socket.

```
close(c);
close(s);
```

11. Stop the program.

Socket Programming — Client Algorithm

1. Start the client program.

2. Create a TCP socket using IPv4.

```
s = socket(AF_INET, SOCK_STREAM, 0);
```

3. Configure the server address with IPv4, port `8080`, and the localhost address `127.0.0.1`.

```
a.sin_family = AF_INET;
a.sin_port = htons(8080);
a.sin_addr.s_addr = inet_addr("127.0.0.1");
```

4. Connect the client socket to the server.

```
connect(s, (struct sockaddr*)&a, sizeof(a));
```

5. Send `"Hello Server"` to the server.

```
send(s, "Hello Server", 12, 0);
```

6. Receive the reply from the server.

```
read(s, buf, sizeof(buf));
```

7. Display the reply received from the server.

```
printf("Server: %s\n", buf);
```

8. Close the client socket.

```
close(s);
```

9. Stop the program.